

Technical Requirements: creating a tool that will be able to make a code review for a proposed merged branch through git. Internal usage for improving code automatization and making sure that code standards match the quality and best practices of implementation.

Proposed AI frameworks to experiment with:

Experiment Example:

A python code for data preprocessing for ingestion into a ML GBM model. Code includes manipulations with dummy data to create a desired structure to be able to put it into the model.

Working files:

1. [dummy_data.csv](#) - created dummy data as an input dataframe into the model.
2. [dummy_generator.py](#) - code of how dummy data was created.
3. [quality_check.py](#) - a code that is checking if final processed data is good for further model ingestion.
4. [github.workflow](#) - integration service between github and openai
5. [break_logic/main.py](#) - code with a wrong final data preprocessing
6. [messy_code/main.py](#) - messy code/ bad practices in terms of naming, comments etc
7. [break_logic_ai_review.txt](#) , [messy_data_ai_review.txt](#) - output of generated ai results for 2 different main.py files during push command in git.

An AI framework should make sure that when the code reviewed it is able:

1. Point out to best practices.
2. Make sure that potential changes do not break existing logic .

For my current task I am using already pretrained AI models that are designed for code review purposes.

AI model	Messy/Bad code Evaluation	Broken Logic Evaluation
ChatGPT 4o mini	AI highlights key improvements for readability and clarity: Naming: Replace unclear class (<code>dp</code>) and method names (<code>p()</code>, <code>g()</code>) with more descriptive names like <code>DataPreprocessor</code>, <code>process_data()</code>, and <code>get_train_test_split()</code>. Parameter and Variable Names: Use explicit names like <code>data</code> instead of <code>d</code>, and rename	AI is able to capture break of the logic and failing of tests. Structure: Good use of classes, methods, and <code>Pipeline</code>, but intentional errors (like feature mismatches) should be removed. Docstrings and Comments: Consistency in language (avoid mixing Russian with English). Intentional Errors: Errors should be removed from preprocessing, as they don't belong in production

	variables such as <code>self.pd</code> to <code>self.processed_data</code>. Code Structure: Add docstrings, error handling, and use meaningful names for pipeline steps (<code>imputer</code>, <code>scaler</code>) and transformers. Refactor: The AI provides an example with clearer naming and improved structure for better readability and maintainability.	code. Error Handling: No error handling or logging, which should be added for better robustness. Flexibility: Make preprocessing steps configurable and support custom target variables.
PolyCoder*	TBD	
DeepCode *	TBD	

*Concept includes testing of other different models with different environments, however due to lack of time, the example is designed only for GPT-4o-mini

Logic of Creating Main.py Messy Code and Broken logic **

Messy Code/ BAd Practices	Broken Logic
1.Poor Naming Conventions: Class names (<code>dp</code>), variables (<code>d</code>, <code>p</code>, <code>a</code>, <code>b</code>, <code>c</code>, <code>e</code>), and function names (<code>process</code>, <code>get_data</code>) are short and non-descriptive. Variable names should give meaningful insights into their purpose (e.g., <code>dp</code> should be <code>DataPreprocessor</code>, <code>d</code> should be <code>data</code>). 2.Lack of Documentation and Comments: No docstrings for most functions. No comments explaining steps or the purpose of specific blocks of code. 3.Poor Code Structure: Hardcoded values (e.g., <code>'target'</code> column). No exception handling (for example, not checking if the CSV file exists or if the data was loaded correctly).	1.Missing Values: The <code>SimpleImputer</code> step is removed from the numeric transformer, so missing values are not filled. 2.Non-Numeric Data: The categorical transformer does not perform one-hot encoding, meaning the categorical features will not be converted to numeric values, which will fail the check for numeric data. 3.Inconsistent Train/Test Feature Sets: A new column <code>extra_feature</code> is added but dropped inconsistently between the training and test sets, causing a mismatch in the number of features between the two sets.

4.Inconsistent Variable Naming: Using <code>pd</code> as a variable name for processed data is confusing because <code>pd</code> is commonly used for importing pandas. The variables <code>a</code>, <code>b</code>, <code>c</code>, and <code>e</code> give no indication of what they represent (training/test features/targets). 5.No Input Validation: There are no checks to verify the input data. This is problematic in case of missing files, invalid data formats, etc.	
Expected Improvements AI should provide	
Use clear and descriptive naming conventions (e.g., <code>DataPreprocessor</code>, <code>features</code>, <code>target</code>, <code>train_features</code>, <code>train_target</code>). Document your code with meaningful docstrings and comments to explain functionality. Handle exceptions and perform validation checks where appropriate.	AI need to define that issue was caused by: - The presence of missing values. - The presence of non-numeric categorical data. -The inconsistency in the number of features between the training and test sets.

Implementation:

Integration with Github and Open AI, triggers prompt every time there is a git push command implemented. Works for any branch that refers to a main one.

For a created repository actions are tracked each time when there is any commit created for existing project . Implemented though github workflows as a separate script within a code.

 **git_al_review** Public

Pin Unwatch 1 Fork

 **test** had recent pushes 1 minute ago

Compare & pull request

main 3 Branches 0 Tags

t Add file <> Code

 **krystyna menkova** and **krystyna menkova** **break_logic** ✓ aefbc4e · 38 minutes ago 22 Commits

 .github/workflows	yaml	3 days ago
 Alchat_GPT4_o_mini	at script	3 days ago
 main.py	break_logic	38 minutes ago
 quality_check.py	ltinial commit	last week

 **README**



menkovakrystyna / git_al_review

Type / to search

<> Code

Issues

Pull requests

Actions

Projects 1

Wiki

Security

Insights

Settings

Actions

New workflow

All workflows

GPT Code Review on Push or Pull Req...

Management

Caches

Attestations

Runners

All workflows

Showing runs from all workflows

Filter workflow runs

Help us improve GitHub Actions

Tell us how to make GitHub Actions work better for you with three quick questions.

Give feedback

24 workflow runs

Event Status Branch Actor

test_branch

GPT Code Review on Push or Pull Request #24: Commit ae7e699 pushed by menkovakrystyna

test

3 minutes ago
22s

...

break_logic

GPT Code Review on Push or Pull Request #23: Commit aefbc4e pushed by menkovakrystyna

main

40 minutes ago
25s

...

messy_code

GPT Code Review on Push or Pull Request #22: Commit a1544dc pushed by menkovakrystyna

main

42 minutes ago
25s

...

Messy Code

11s

```
ChatCompletionMessage(content='Your code poses several issues that impact readability, maintainability, and overall design. Here's a detailed review along with suggestions for improvement:\n\n#### Class and Method Naming\n1. **Class Name** `dp` **: \n - The name `dp` is ambiguous and does not convey any meaning about its purpose. A more descriptive name such as `DataPreprocessor` or `DataProcessor` would be more suitable.\n2. **Method Names** **: \n - Methods like `p()` and `g()` are not descriptive. Use names like `process_data()` and `get_train_test_split()` to enhance clarity.\n\n#### Parameter Naming\n3. **Unclear Parameter Names** **: \n - The parameter `d` in `__init__` should be renamed to something more explicit, like `data` or `input_dataframe`, which would improve readability.\n\n#### Variable Names\n4. **Variable Naming** **: \n - Internal variable names such as `self.d`, `self.pd`, `self.a`, `self.b`, `self.c`, and `self.e` lack meaning. Rename `self.pd` to `self.processed_data`, and for the other variables, use names like `X_train`, `X_test`, `y_train`, and `y_test` to represent their respective data.\n\n#### Unclear Method Implementation\n5. **Docstrings** **: \n - Each method should contain a docstring explaining its functionality, parameters, and return values. This documentation is essential for understanding the code's purpose.\n6. **Error Handling** **: \n - The method `p()` does not contain any error handling for potential issues, such as missing columns in the input DataFrame. Add checks to validate inputs before processing.\n\n#### Code Structure\n7. **Pipeline Steps** **: \n - The variable names for each step in the pipelines (`i`, `s`, `o`, etc.) should be full words. For example, use `imputer` and `scaler` instead of `i` and `s` for clarity.\n8. **Transformers in ColumnTransformer** **: \n - Similar to the pipelines, the names for the transformers (`n`, `c`) should explicitly describe what they represent, such as `numerical_transformer` and `categorical_transformer`.\n\n#### Output\n9. **Output Variable Naming** **: \n - Rename `Xp` to `X_processed` for clarity on what the variable represents.\n10. **Returned Variables** **: \n - In the `g()` method, instead of returning `self.a`, `self.b`, `self.c`, and `self.e`, return `X_train`, `X_test`, `y_train`, and `y_test`.\n\n#### Main Executable\n11. **Main Execution Block** **: \n - The variable `d` for reading data should be named `data` for clearer semantics. Additionally, include comments describing each step's purpose to provide context.\n\n12. **Unused Variables** **: \n - If you utilize `evaluate_preprocessed_data`, ensure it's clear what the function does and what outputs it expects. Comments may be necessary here as well.\n\n### Example Refactor\nHere's a refactored version of your code with the above suggestions implemented:\n\n```python\nimport pandas as pd\nfrom sklearn.preprocessing import StandardScaler, OneHotEncoder\nfrom sklearn.compose import ColumnTransformer\nfrom sklearn.pipeline import Pipeline\nfrom sklearn.impute import SimpleImputer\nfrom quality_check import evaluate_preprocessed_data\n\nclass DataPreprocessor:\n    def __init__(self, input_dataframe):\n        self.input_dataframe = input_dataframe\n        self.processed_data = None\n        self.X_train = None\n        self.X_test = None\n        self.y_train = None\n        self.y_test = None\n\n    def process_data(self):\n        \"\"\"\n        Applies transformations, and splitting into train/test sets.\n        \"\"\"\n        X = self.input_dataframe.drop('target', axis=1)\n        y = self.input_dataframe['target']\n\n        numerical_features = X.select_dtypes(include=[\"int64\", \"float64\"]).columns\n        categorical_features = X.select_dtypes(include=[\"object\"]).columns\n\n        numerical_transformer = Pipeline(steps=[\n            ('imputer', SimpleImputer(strategy='mean')),\n            ('scaler', StandardScaler())\n        ])\n        categorical_transformer =
```

Broken Logic

6s

```

29 ChatCompletionMessage(content='The provided code has several noteworthy elements, but there are significant issues, especially related to intentional flaws introduced in the data preprocessing pipeline. Below is a detailed review that addresses the code's structure, functionality, and design, along with suggestions for improvement.\n\n### Code Review\n\n1. Code Structure and Organization: The overall structure of the classes and methods is good. The separation of concerns (data loading, preprocessing, and evaluation) is clear and well-defined.\n   - The use of 'Pipeline' and 'ColumnTransformer' is appropriate for handling different types of data.\n\n2. Docstrings and Comments: The code includes docstrings and comments that explain the purpose and functionality of methods and transformations, which is good practice for maintainability and readability.\n   - However, parts of the docstrings are in Russian, which could be inconsistent if the majority of your codebase is in English. It's better to maintain a consistent language throughout.\n\n3. Intentionally Induced Errors: It is concerning that the code has intentional errors ("Breaking imputation logic" and "Intentionally creating mismatch in features"), including the removal of one-hot encoding and adding an extra feature to break the pipeline. This seems counterproductive for a preprocessing script. It's crucial that preprocessing aims to prepare data for machine learning effectively, not to create errors.\n   - Introducing errors for testing purposes should ideally be done in a controlled manner (like unit tests) rather than in a production-level preprocessing pipeline.\n\n4. Feature Mismatch: The comment explaining how a feature mismatch is created by not dropping the extra column in the test set but dropping it in the training set is essential to note. Still, this approach should only be used in a testing or debugging context and not in functional preprocessing.\n\n5. Placeholder Comments and Incomplete Logic: The comment above the missing imputation step could mention why it is being removed or what the intended behavior should be instead.\n   - The 'evaluate_preprocessed_data' function call may fail as intended in the code, but if this is part of a debugging effort, this workflow should be documented explicitly.\n\n6. Error Handling: The code does not include any error handling or logging. It would be beneficial to add exception handling around critical operations (e.g., reading the CSV file, transforming data) to provide informative error messages.\n\n7. Flexibility and Reusability: Consider making the preprocessing logic (like imputation strategies) configurable, allowing users to pass parameters based on the data they are working with.\n   - Rather than a fixed column name 'target', allow users to specify the target variable. This would enhance the reusability of the class.\n\n8. Encapsulation: All attributes in the 'DataPreprocessor' class are public. It might be good to make some of them private (e.g., 'self.data', 'self.preprocessed_data') to encapsulate the internal state better.\n\n### Suggested Improvements\n\n1. Remove Intentional Errors: As a best practice, remove the intentional mismatches and errors from the code. Instead, consider writing unit tests that confirm the expected behavior of your preprocessing steps.\n\n2. Enhance Flexibility: Allow users to set parameters, such as which columns to drop, the target variable, and imputation strategies.\n\n3. Integrate Logging: Add logging to capture events, especially during data loading and transformation stages.\n\n4. Error Handling: Implement try-except blocks where necessary to catch exceptions and provide clear feedback on what went wrong.\n\n5. Follow a Consistent Language and Style Guide: Ensure all comments and documentation are in one language and conform to a consistent style guide to improve readability and maintainability.\n\n6. Unit Testing: Implement some unit tests for the 'DataPreprocessor'

```

Link to github:

https://github.com/menkovakrystyna/git_al_review

Used prompt: **Please review the following code:** `\n{code_content}`

Conclusion:

For both problems gpt40-mini handles properly for identifying existing problems and giving some recommendations.

Further Usage:

Service can be used both:

- Virtually . Through Github Artifacts file can be saved and accessed. Further work with this file can be addressed through optimizing prompt itself and making it more specific. Prompt can be written in a way to pass certain custom criterias and once code is succeeded AI review code indicated about it.
- Offline. I created a similar local version that can check the code locally before making a commit, that will help to get into better practices before making a submission.

Detailed pipeline of the process and infrastructure:

1. **Event Trigger:** Developer opens a pull request.
2. **GitHub Actions:** Workflow is triggered.
3. **API Call:** GitHub sends the code to the AI model's API for review.
4. **AI Processing:** AI reviews the code and generates suggestions or comments.
5. **Feedback:** The AI model sends back feedback. Can be stored as logs, or in any cloud storage if required.

Any additional propositions about infrastructure can not be proposed, as there is a requirement for getting more detailed specification of the data, existing infrastructure and main purposes of the service.

